SOFTWARE

DESIGN

DOCUMENT

CAREER CRAFT

Prepared by

P Yedukrishna

Rayan Vincent

Sreejith v m

Subin K Thomas

Guide

Mrs. Libsy Ann Merin Baby

15 OCTOBER 2025

Contents

1 1.Overview

1.1 Scope

Career Craft is an AI-powered web application designed to deliver a comprehensive placement preparation experience for engineering students. The scope includes mock interview simulations using AI-driven tools for speech-to-text and feedback, resume building with NLP-based parsing and ATS optimization via cosine similarity, adaptive technical quizzes on DSA and databases with IRT and Chart.js analytics, soft skills heatmaps using sentiment analysis and speech evaluation, skill gap analysis comparing user data to job requirements, domain-specific preparation with tailored quizzes and roadmaps, real-time notifications via Socket.IO for alerts and milestones, and student performance tracking with scikit-learn ML and visualizations. The project utilizes open-source AI/NLP/ML tools for cost-effective, scalable solutions, deployable locally (SQLite, Flask) or on free cloud platforms, with optional enhancements, ensuring students are well-prepared for diverse industry roles.

1.2 Purpose

The purpose of this document is to provide a comprehensive software design documentation of Career Craft, outlining its architecture, core components, data structures, and implementation strategies. This document serves as a guide for:

- Software implementation, ensuring structured and modular development.
- Testing and validation, focusing on performance optimization and user experience refinement.
- AI/ML integration and real-time data processing to ensure seamless functionality.
- UI/UX design principles, ensuring an intuitive and adaptive user experience.

1.3 Intended Audience

- Development Team
- Project Owner
- Project Mentor
- Members of VJCET
- Testers & End-Users

2 Definitions, Acronyms and Abbreviations

This section defines key terms, acronyms, and abbreviations used in this document:

Term	Definition
AI	Artificial Intelligence – Technology for adaptive learning and analysis.
ML	Machine Learning – Algorithms for data-driven performance improvement.
NLP	Natural Language Processing – Techniques for text and speech analysis.
ATS	Applicant Tracking System – Software for screening resumes.
API	Application Programming Interface – Tools for software integration.
SDD	Software Design Document – Document detailing system design.
SRS	Software Requirements Specification – Document for system requirements.
UML	Unified Modeling Language – Standard for software design diagrams.
UI	User Interface – Interactive layer for user engagement.
SDK	Software Development Kit – Tools for application development.
JWT	JSON Web Token – Secure method for authentication.
Socket.IO	Real-time communication library for web applications.
Nodemailer	Email sending library for Node.js.
MongoDB	NoSQL database for flexible data storage.
React.js	JavaScript library for building user interfaces.
Node.js	JavaScript runtime for server-side execution.
Chart.js	JavaScript library for data visualization.
Soft Skills	Non-technical skills like communication and leadership.
Aptitude	Reasoning and problem-solving abilities.
Mock Interview	Simulated interview for practice with feedback.
Skill Gap	Difference between required and current skills.

3 Conceptual Model for Software Design Documentation

3.1 Software Design in Context

- React.js – Front-end framework.
- Node.js/Express.js – Back-end server.
- MongoDB – Database management.
- AI/ML Libraries (e.g., Python with NLTK, scikit-learn) – For analysis and predictions.

3.2 Software Design Descriptions within the Life Cycle

3.2.1 Influences on SDD Preparation

The preparation of the Software Design Description (SDD) for Career Craft has been influenced by several factors:

- **Personalized AI Experience**: The design heavily considers AI-specific requirements such as adaptive quizzes, real-time feedback, and NLP for resumes and interviews.
- **Real-Time Notifications**: Ensuring seamless communication via Socket.IO and Nodemailer impacts design choices, particularly in event handling and user alerts.
- **User Accessibility and Scalability**: The design process emphasizes cost-effective deployment, open-source models, and scalability for large student cohorts.
- **Module Diversity**: The design must support modular systems for quizzes, interviews, and analytics to allow expansion with new features.
- **Competitive Preparation**: The platform's focus on skill gaps and personalized roadmaps influences the core design documentation.

3.2.2 Influences on Software Life Cycle Products

The design decisions documented in the SDD influence several other software life cycle products:

- **Requirements Specification**: The SDD is directly shaped by the functional and non-functional requirements specified for the platform, ensuring alignment between intended features and implemented design.
- **System Architecture**: AI integration, real-time notifications, and data analytics form critical architectural components documented in the SDD.
- **Testing and Validation Plans**: Design choices regarding mock interviews, quizzes, and skill analysis influence the creation of specific test cases and performance benchmarks.
- **User Manuals and Tutorials**: The adaptive learning and module interactions documented in the SDD form the foundation for user training materials.

3.2.3 Design Verification and Design Role in Validation

The Software Design Description plays a crucial role in both design verification and product validation:

- **Design Verification**:
  - Each design component (AI mock interviews, resume builder, analytics) is verified against the requirements stated in the SDD.
  - Peer reviews and design walkthroughs ensure alignment between conceptual design and implementation.
  - Simulation tools and prototyping are used to verify complex components such as NLP and ML models before full implementation.
- **Design Role in Validation**:
  - The SDD serves as the reference document for acceptance testing, ensuring that all features and behaviors match the intended design.
  - Usability testing with real students ensures that the adaptive experience and analytics meet user expectations.
  - Continuous feedback loops, informed by early prototypes and user testing, allow the design to evolve while retaining alignment with the original SDD.

4 Design Documentation Information Content

4.1 Introduction

This section outlines the key elements of the design documentation for Career Craft, an AI-powered web application for campus placement preparation. It covers the high-level architecture, data flow diagrams illustrating interactions between frontend, backend, and AI modules, functional components such as mock interview simulators and skill gap analyzers, and key design principles including modularity, scalability, and security. These elements ensure that the design aligns with the functional requirements from the SRS, such as adaptive quizzes and real-time notifications, while emphasizing open-source tools for cost-effectiveness and ease of deployment.

4.2 SDD Identification

The Software Design Document (SDD) for Career Craft follows the IEEE standard for software design descriptions, which provides a structured framework for documenting design decisions, viewpoints, and rationales. This adherence ensures traceability from requirements to implementation, facilitating verification, validation, and maintenance.

4.3 Design Stakeholders

The design process involves multiple stakeholders whose needs and perspectives shape the system's architecture and features:

- **Developers**: Responsible for implementing modules like the backend APIs and AI integrations, requiring clear component diagrams and code guidelines.
- **Project Owner**: Oversees alignment with business goals, such as affordability and scalability for student users.
- **Mentor**: Provides technical oversight, ensuring best practices in AI/ML deployment and security.
- **Faculty Evaluators**: Assess compliance with educational standards and usability for campus environments.
- **Testers & End-Users**: Students and testers who validate user experience, focusing on intuitive interfaces and accurate feedback mechanisms.

4.4 Design Views

The design views provide multifaceted perspectives on the system to support analysis and implementation:

- **Context View**: Depicts external interactions, such as student inputs and external services.
- **Composition View**: Illustrates how modules like the resume builder and quiz platform are composed and deployed.
- **Logical View**: Details object-oriented structures, including classes for users and analyzers.

- **Dependency View**: Maps relationships, e.g., AI modules depending on backend processing.
- **Information View**: Describes data flows, from user states to AI-generated predictions.
- **Interface View**: Focuses on user and system interfaces, ensuring responsive web elements.
- **Interaction View**: Shows sequence of events, like quiz submission to feedback delivery.
- **State Dynamics View**: Models transitions, such as from ”Preparing” to ”Analyzed” user states.

4.5 Design Overlays

Overlays address cross-cutting concerns that influence multiple views:

- **Performance**: Ensures low-latency processing for real-time features, targeting 5-second load times and 1-second query responses.
- **Network**: Supports scalable deployment on cloud platforms like Render or Vercel, with Socket.IO for bidirectional communication.
- **Security**: Implements JWT authentication, data encryption, and NLP moderation to protect sensitive student information.
- **User Experience**: Prioritizes intuitive dashboards and adaptive interfaces, learnable in under 15 minutes, with visualizations via Chart.js.

4.6 Design Rationale

The design adheres to object-oriented (OO) principles to promote reusability, maintainability, and extensibility, particularly for evolving features like domain-specific roadmaps:

- **Encapsulation**: Bundles data and methods in modules (e.g., AI analyzer hides complex NLP logic), reducing complexity.
- **Inheritance**: Allows subclasses for quiz types (e.g., technical vs. aptitude), enabling shared behaviors.
- **Polymorphism**: Supports varied inputs (speech/text) in the mock interview module via unified interfaces.
- **Event-Driven**: Uses observers for triggers like quiz completion, ensuring loose coupling in notification flows.

4.7 Design Languages

The following languages and notations are employed for precise expression and interoperability:

- **UML**: For structural and behavioral diagrams, providing visual clarity on system interactions.
- **JavaScript**: Core for frontend (React.js) and backend (Node.js) logic, ensuring dynamic, event-driven behaviors.
- **JSON/XML**: For data serialization in APIs and configurations, facilitating seamless exchange between components.
- **Python**: For AI/ML scripts (e.g., spaCy for NLP), leveraging libraries like scikit-learn for analysis.

5 Design Viewpoints

5.1 Introduction

Career Craft is an AI-driven platform for campus placement preparation, integrating technical quizzes, mock interviews, resume analysis, and soft skills tracking to empower engineering students. It leverages open-source tools for affordability and scalability, deployable on local or cloud environments. The design viewpoints—aligned with IEEE 1016-2009—offer a holistic examination of the architecture, from external contexts to internal dynamics, ensuring the system meets SRS requirements like real-time feedback and personalized roadmaps while supporting future expansions such as additional domains.

5.2 Context Viewpoint

5.2.1 Design Concerns

- Real-time feedback in AI modules, ensuring low-latency responses for interviews and quizzes.
- Scalability for multiple users and data processing, handling concurrent sessions without performance degradation.
- Integration of AI/ML for personalization, incorporating tools like BERT for domain-specific content.
- Enhancing user experience through adaptive learning and notifications, with Socket.IO for instant updates.

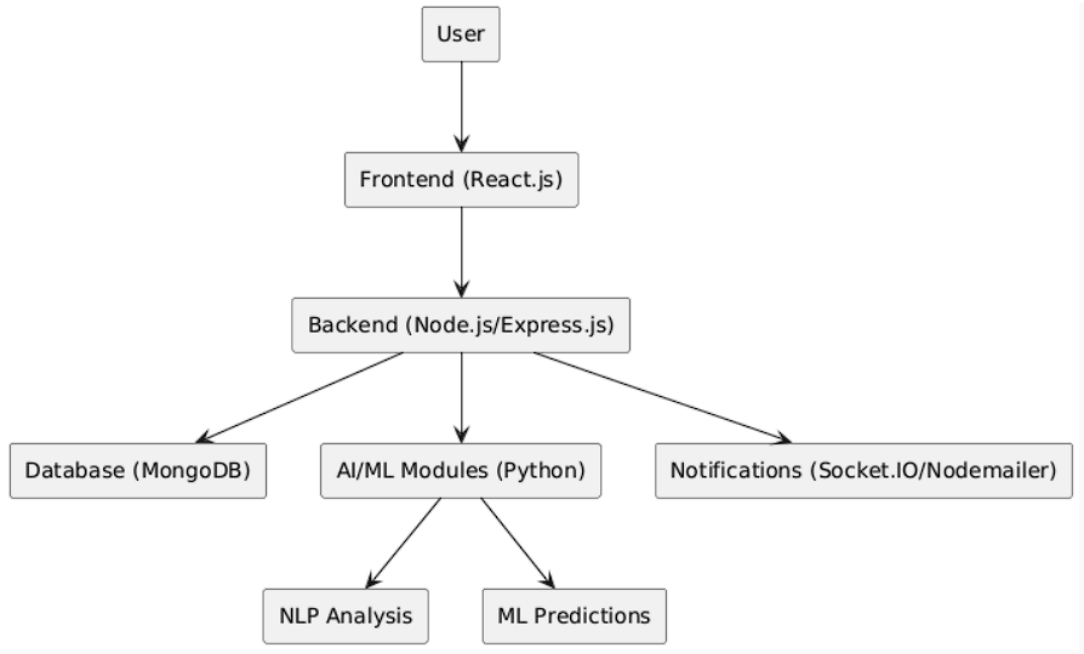

Figure 5.1: Block Diagram of Career Craft

5.2.2 Design Elements

- **Students (Actors)**: Primary users providing inputs via quizzes, resumes, and interviews.
- **System Boundaries**: Encompasses web app, AI servers for analysis, and database for persistence.
- **External Interfaces**: Handles user inputs, email via Nodemailer, and web scraping for company data.
- **Events**: Triggers like quiz completion, skill analysis, and notifications drive dynamic flows.

5.2.3 Example Languages

- UML for high-level modeling of boundaries and actors.
- JSON for configuration data in external API calls.
- JavaScript for core logic in handling user events.
- Python for AI/ML processing of external data streams.

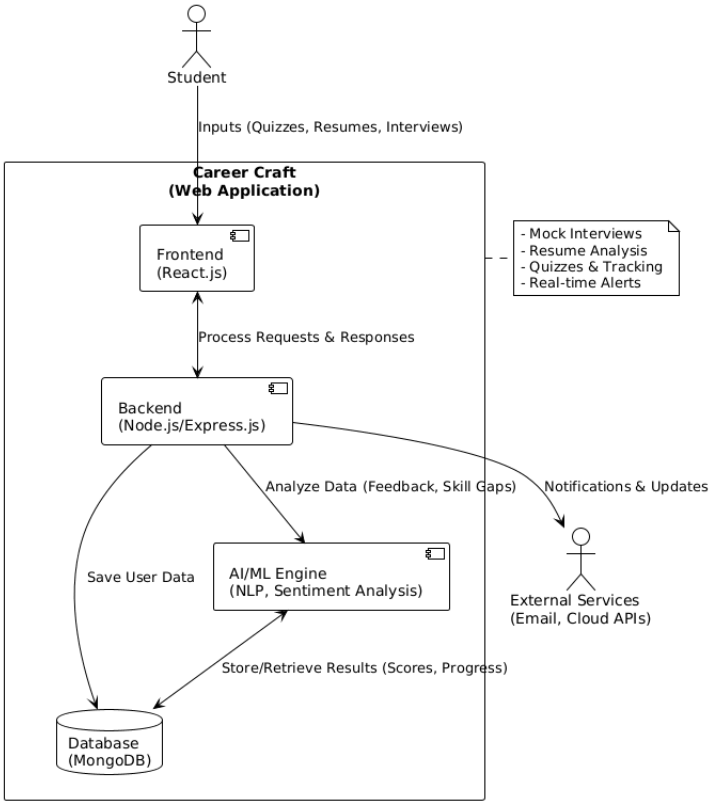

Figure 5.2: Context Diagram of Career Craft

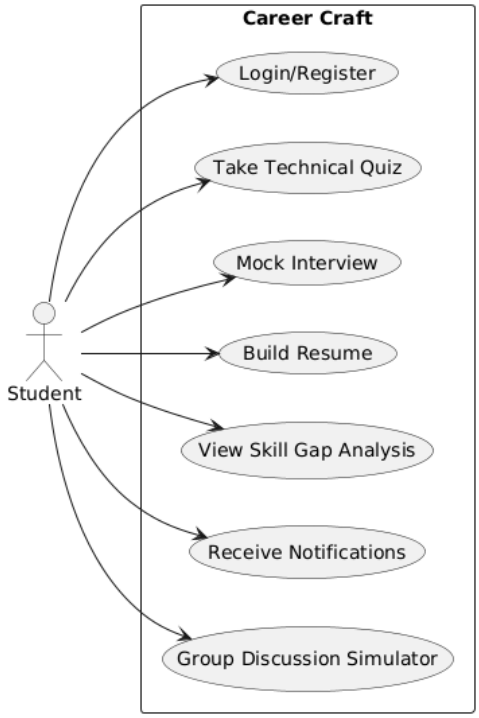

Figure 5.3: Use Case Diagram of Career Craft

5.3 Composition Viewpoint

5.3.1 Design Concerns

- Modular separation of core functionalities (mock interviews, quizzes, analytics) to enable independent development and testing.
- Communication between components (e.g., user input to AI analysis) via APIs, ensuring loose coupling and fault isolation.

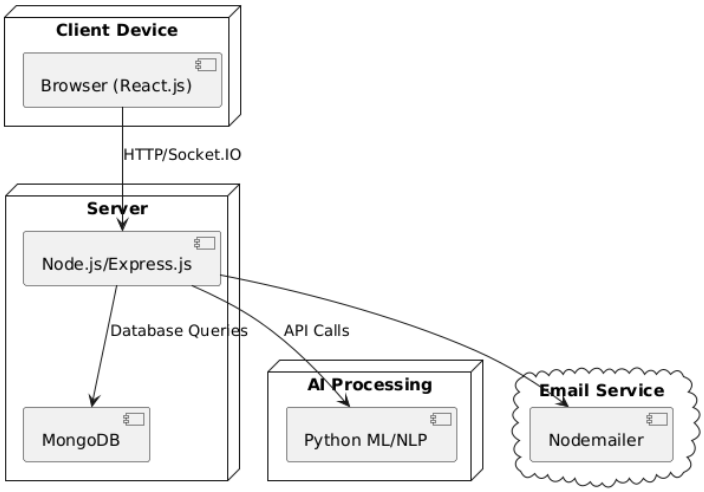

Figure 5.4: Deployment Diagram of Career Craft

5.3.2 Design Elements

- **AI Mock Interview Module**: Handles speech-to-text and feedback using Whisper and BERT.
- **Resume Builder Module**: Processes uploads with spaCy for ATS optimization.
- **Quiz Platform Module**: Manages adaptive quizzes with IRT algorithms.
- **Performance Analyzer Module**: Tracks trends using scikit-learn.
- **Notification Provider Module**: Delivers alerts via Socket.IO.

5.3.3 Example Languages

- UML component diagrams for visualizing module interconnections.
- JavaScript for system communication through RESTful APIs.

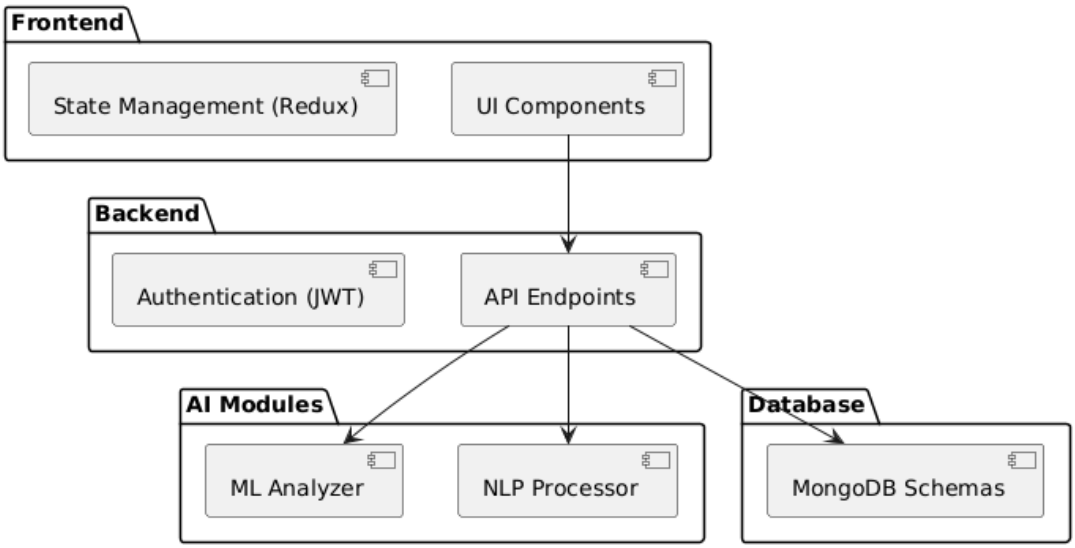

Figure 5.5: Component Diagram of Career Craft

5.4 Logical Viewpoint

5.4.1 Design Concerns

- Object-oriented structure for platform entities to support extensibility, e.g., adding new quiz types.
- Clear separation of concerns for better maintainability, isolating UI from business logic.

5.4.2 Design Elements

- **User Class**: Encapsulates profile data, preferences, and progress.
- **Quiz Class**: Defines question banks, scoring, and adaptive logic.

- **Interview Class**: Manages audio inputs and sentiment analysis.
- **Analyzer Class**: Processes data for skill gaps and roadmaps.

5.4.3 Example Languages

- UML class diagrams for entity relationships and attributes.
- JavaScript code for class implementation in Node.js and React.js.

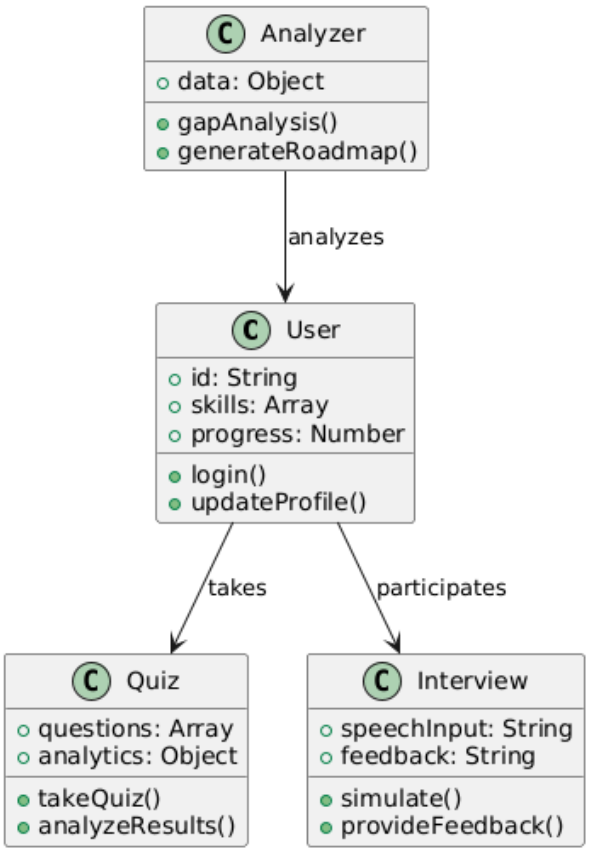

Figure 5.6: Class Diagram of Career Craft

5.5 Dependency Viewpoint

5.5.1 Design Concerns

- Tracking dependencies between platform systems (e.g., AI analysis depending on user input) to avoid circular references.
- Minimizing tight coupling between systems, using interfaces for decoupling modules like notifications from analyzers.

5.5.2 Design Elements

- **User Input Handler → AI Analyzer**: Forwards resumes and responses for NLP processing.

- **Notification Provider → Performance Analyzer**: Triggers alerts based on tracking results.
- **Database Manager → All Modules**: Provides centralized data access with caching for efficiency.

5.5.3 Example Languages

- UML dependency diagrams to visualize flow directions.
- JavaScript modules with import/export for runtime dependencies.

5.6 Information Viewpoint

This viewpoint defines the flow of data and state information during usage, ensuring secure and efficient handling across components. Critical elements include:

- **User State**: Tracks progress, skills, and confidence levels, updated via backend APIs.
- **Quiz/Interview Data**: Includes scores, feedback, and analytics, processed by AI for personalization.
- **Notification State**: Manages alerts and updates, queued in the database for delivery.
- **AI Packets**: Carries NLP results (e.g., sentiment scores) and ML predictions (e.g., skill gaps) between modules.

Data flows adhere to SRS performance requirements, with encryption for sensitive elements like resumes.

5.7 Patterns Use Viewpoint

The design leverages proven design patterns to address common challenges in AI-integrated systems:

- **Singleton**: For Session Manager, ensuring centralized control of user sessions across distributed components.
- **Observer**: For event handling (e.g., quiz completion triggers notification), promoting decoupling in real-time features.
- **Factory**: For module creation (different quiz types with distinct attributes), allowing dynamic instantiation based on domain selection.

These patterns enhance scalability and maintainability, aligning with non-functional requirements like 99.9% availability.

5.8 Interface Viewpoint

5.8.1 Design Concerns

- Ensuring intuitive web interfaces with responsive design for diverse devices.
- Providing clear feedback via dashboards and visualizations to build user confidence.
- Supporting adaptive inputs for accessibility.

5.8.2 Design Elements

- **Dashboard**: Displays progress, skill gaps, and notifications with Chart.js graphs.
- **Input Handler**: Processes speech/text in interviews, integrating with Whisper for transcription.
- **Menut**: Allows module selection and domain choices, with Tailwind CSS for styling.

Interfaces prioritize usability, as per SRS, with JWT-secured access.

5.9 Structure Viewpoint

5.9.1 Design Concerns

- Maintaining a well-defined package structure to organize code for collaborative development.
- Separating front-end, back-end, and AI handling into distinct modules for independent scaling.

5.9.2 Design Elements

- **Front-End (UI Logic)**: React.js components for interactive views like quiz interfaces.
- **Back-End (API, Database)**: Node.js/Express.js for endpoints and MongoDB interactions.
- **AI/ML Integration (Analysis, Models)**: Python scripts invoked via APIs for NLP and ML tasks.

5.9.3 Example Languages

- UML package diagrams for hierarchical organization.

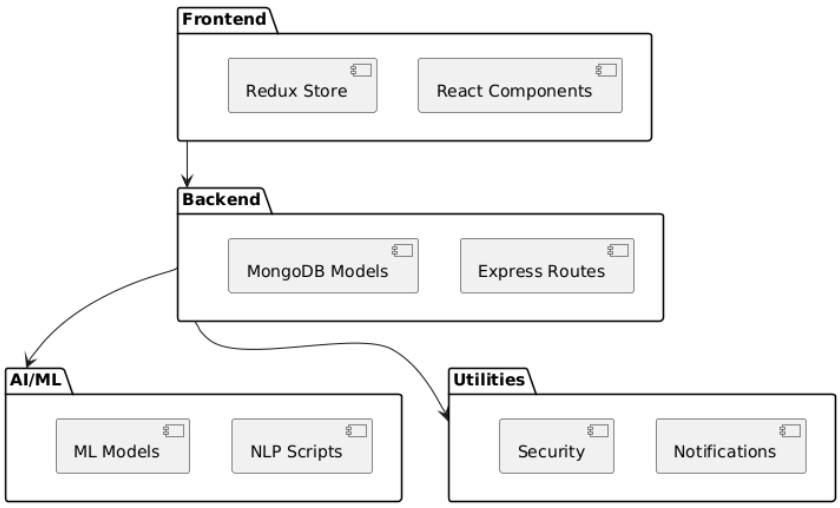

Figure 5.7: Package Diagram of Career Craft

5.10 Interaction Viewpoint

5.10.1 Design Concerns

- Capturing real-time interactions between users and platform modules, such as seamless quiz-to-feedback flows.
- Handling events in a clear, traceable way to debug issues in dynamic scenarios like live interviews.

5.10.2 Design Elements

- **User Starts Quiz**: Triggers adaptive question loading and progress update.
- **User Submits Resume**: Initiates NLP parsing and ATS optimization.
- **System Analyzes Performance**: Computes scores and generates roadmaps via ML.

5.10.3 Example Languages

- UML sequence diagrams for step-by-step event flows.
- JavaScript async/await for handling asynchronous API calls.

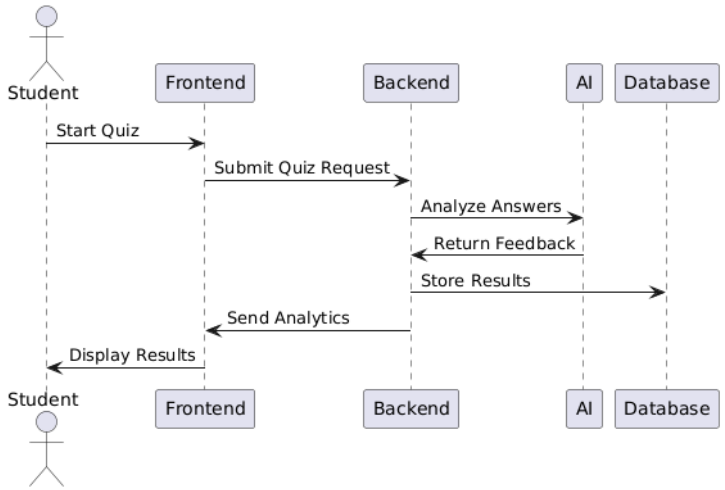

Figure 5.8: Sequence Diagram of Career Craft

5.11 State Dynamics Viewpoint

5.11.1 Design Concerns

- Defining clear state transitions for users and modules to prevent invalid states, e.g., during analysis.
- Handling special states (analysis in progress, notification sent) with timeouts for reliability.

5.11.2 Design Elements

- **User States**: Registered → Preparing → Analyzed → Notified, with guards for incomplete data.
- **Module States**: Idle → Processing → Feedback Given, reverting on errors.

This ensures robustness, aligning with SRS safety requirements for error handling.

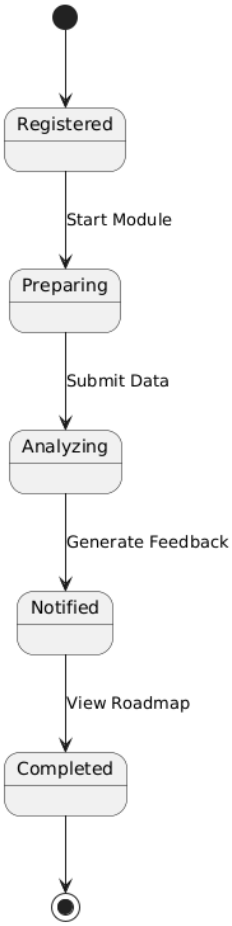

Figure 5.9: State Machine Diagram of Career Craft

6 Conclusion

This Software Design Documentation (SDD) for Career Craft serves as a comprehensive guide for development, testing, and future updates. It ensures that both technical and design-level decisions are properly documented, allowing future developers to understand, maintain, and expand the system efficiently. The SDD provides a detailed blueprint of the system's architecture, including its modular components, data flows, and interaction patterns, which facilitates seamless collaboration among the development team and stakeholders. Additionally, it establishes a foundation for ongoing enhancements by outlining scalability considerations and integration points for emerging AI/ML technologies. By adhering to IEEE standards, the document ensures consistency and traceability, enabling rigorous testing and validation processes to meet user expectations and performance benchmarks. Future iterations of Career Craft can leverage this SDD to incorporate user feedback, optimize resource usage, and address evolving industry requirements, ensuring the platform remains a cutting-edge solution for campus placement preparation.